Priority Arbiter with Aging: Code Breakdown and Testbench Results

Reconfigurable Computing, AY 2026-27 | Simulation of the arbitration core, no physical sensors involved

1. Overview

This report covers the four Verilog files that make up the arbitration-core simulation, explains each one in detail, and then breaks down what the testbench output and the Vivado waveform mean. The goal was to prove in simulation, before touching any hardware, four things:

1. The higher-priority requester wins when both ask at once.
2. Plain fixed priority starves the low-priority requester under heavy load.
3. Aging bounds the low-priority wait without breaking priority.
4. No interrupt is ever lost and the shared resource is never double-granted.

The simulation ran 60,000 clock cycles (600 microseconds at 100 MHz) and all **50 checks passed**, with identical output in Icarus Verilog and in Vivado XSim.

1.1 The four files

File	Role	Synthesizable
arbiter.v	The design under test: priority arbiter with aging	Yes
synth_requester.v	Synthetic interrupt generator plus a fake sensor service model	No (simulation only)
arb_system.v	Wrapper: arbiter, two generators, latency trackers and protocol checkers	No (simulation only)
tb_arbiter.v	Testbench: eight experiments run side by side, self-checking	No (simulation only)

1.2 How they connect

```
                tb_arbiter  (clock, reset, final checks)
                     |
      eight copies: s1 ... s8  =  arb_system
                     |
   +------------------ arb_system -------------------+
   |  synth_requester src0 --req[0]--+                |
   |        ^ grant[0], done[0]      +--> arbiter      |
   |  synth_requester src1 --req[1]--+                |
   |  lat_tracker lat0, lat1 : request-to-grant wait   |
   |  always-on checkers     : 4 error counters        |
   +---------------------------------------------------+
```

1.3 The handshake every requester follows

Signal	Direction	Meaning
req[i]	requester to arbiter	Held high from the moment data is pending until the service finishes
grant[i]	arbiter to requester	Registered, one-hot. Held until the owner pulses done
done[i]	requester to arbiter	One-cycle pulse when the service is complete

The real ACL2 and HYGRO controllers will use exactly this interface, so arbiter.v will not need to change when the fake sensors are replaced.

1.4 How to run

```
iverilog -g2005 -Wall -o sim_arbiter arbiter.v synth_requester.v arb_system.v
tb_arbiter.v
vvp sim_arbiter
```

In Vivado: add the four files as simulation sources, set tb_arbiter as the simulation top, and run for at least 601 microseconds. The PASS and FAIL lines appear in the Tcl console.

1.5 Units used throughout

One cycle is 10 ns at 100 MHz. Latency means the number of cycles a request was visible but not yet granted. The minimum possible latency is 1 (request seen in one cycle, grant appears in the next).

2. arbiter.v (the design under test)

```
`timescale 1ns/1ps
//
================================================================================
// arbiter.v : 2-requester, non-preemptive, priority arbiter with aging
//
// Policy
//   effective_priority(i) = base_prio(i) + bonus(i)
//   bonus(i) grows by 1 every AGE_THRESH cycles that request i has been
//   waiting (req=1, not granted), saturating at 2^BONUS_W-1.
//   bonus is cleared when i is granted or drops its request.
//   Highest effective priority wins. TIE -> the requester that was NOT
//   served last (least-recently-served). After reset, channel 0 wins ties.
//   aging_en=0  -> bonus stays 0 -> plain fixed-priority arbiter (baseline).
//
// Handshake (same for every requester, so real sensor controllers drop in)
//   req[i]   : held high by requester i from "data pending" until it
finishes
//   grant[i] : registered, one-hot; held until done[owner] pulses
//   done[i]  : 1-cycle pulse from requester i when its service is complete
//
// Non-preemptive: once granted, a requester keeps the grant until its done,
// no matter what the priorities do meanwhile (SPI/I2C frames can't be
aborted).
// Decision latency: req seen in IDLE -> grant visible the next cycle.
// Between two services the arbiter spends 1 IDLE cycle (grant low).
//
// Synthesizable Verilog-2001. Synchronous active-high reset.
//
================================================================================
module arbiter #(
    parameter PRIO_W  = 4,                      // base priority width
    parameter BONUS_W = 8,                      // aging bonus width
    parameter THR_W   = 16,                     // aging threshold width
    parameter EFF_W   = ((BONUS_W > PRIO_W) ? BONUS_W : PRIO_W) + 1  // do
```

```
not override
)(
    input  wire                clk,
    input  wire                rst,
    input  wire [1:0]          req,
    input  wire [1:0]          done,
    output reg  [1:0]          grant,
    // run-time configuration
    input  wire                aging_en,
    input  wire [THR_W-1:0]    age_thresh,       // cycles of waiting per +1
bonus (>=1)
    input  wire [PRIO_W-1:0]   base_prio0,
    input  wire [PRIO_W-1:0]   base_prio1,
    // visibility for debug / waveforms
    output wire [EFF_W-1:0]    eff_prio0,
    output wire [EFF_W-1:0]    eff_prio1
);
    localparam S_IDLE = 1'b0;
    localparam S_BUSY = 1'b1;
    localparam [BONUS_W-1:0] BONUS_MAX = {BONUS_W{1'b1}};

    reg                state;
    reg                owner;          // channel currently holding the grant
    reg                last_served;    // channel granted most recently
    reg [THR_W-1:0]    wait0, wait1;   // cycles since last bonus step
    reg [BONUS_W-1:0]  bonus0, bonus1; // aging bonus

    assign eff_prio0 = {{(EFF_W-PRIO_W){1'b0}},  base_prio0} + {{(EFF_W-
BONUS_W){1'b0}}, bonus0};
    assign eff_prio1 = {{(EFF_W-PRIO_W){1'b0}},  base_prio1} + {{(EFF_W-
BONUS_W){1'b0}}, bonus1};

    // ---------------- combinational winner selection ----------------
    reg win;          // which channel would win right now
    reg win_valid;    // at least one request pending
    always @* begin
        win_valid = |req;
        win       = 1'b0;
        case (req)
            2'b01:   win = 1'b0;
            2'b10:   win = 1'b1;
            2'b11:   begin
                if      (eff_prio0 > eff_prio1) win = 1'b0;
```

```
                else if (eff_prio1 > eff_prio0) win = 1'b1;
                else                            win = ~last_served;   // tie-
break
            end
            default: win = 1'b0;
        endcase
    end

    // ---------------- state, grant and aging ----------------
    always @(posedge clk) begin
        if (rst) begin
            state       <= S_IDLE;
            owner       <= 1'b0;
            last_served <= 1'b1;        // so channel 0 wins the very first
tie
            grant       <= 2'b00;
            wait0 <= {THR_W{1'b0}};  wait1 <= {THR_W{1'b0}};
            bonus0 <= {BONUS_W{1'b0}}; bonus1 <= {BONUS_W{1'b0}};
        end else begin
            // ---- aging counters: only while waiting (req && !grant) ----
            if (!req[0] || grant[0] || !aging_en) begin
                wait0 <= {THR_W{1'b0}}; bonus0 <= {BONUS_W{1'b0}};
            end else if (({1'b0, wait0} + 1) >= {1'b0, age_thresh}) begin
                wait0 <= {THR_W{1'b0}};
                if (bonus0 != BONUS_MAX) bonus0 <= bonus0 + 1'b1;
            end else
                wait0 <= wait0 + 1'b1;

            if (!req[1] || grant[1] || !aging_en) begin
                wait1 <= {THR_W{1'b0}}; bonus1 <= {BONUS_W{1'b0}};
            end else if (({1'b0, wait1} + 1) >= {1'b0, age_thresh}) begin
                wait1 <= {THR_W{1'b0}};
                if (bonus1 != BONUS_MAX) bonus1 <= bonus1 + 1'b1;
            end else
                wait1 <= wait1 + 1'b1;

            // ---- grant FSM (later assignments override the aging ones) ---
-
            case (state)
                S_IDLE: begin
                    if (win_valid) begin
                        grant       <= win ? 2'b10 : 2'b01;
                        owner       <= win;
```

```
                        last_served <= win;
                        state       <= S_BUSY;
                        if (win) begin wait1 <= {THR_W{1'b0}}; bonus1 <=
{BONUS_W{1'b0}}; end
                        else     begin wait0 <= {THR_W{1'b0}}; bonus0 <=
{BONUS_W{1'b0}}; end
                    end
                end
                S_BUSY: begin
                    if (done[owner]) begin      // only the owner can release
                        grant <= 2'b00;
                        state <= S_IDLE;
                    end
                end
            endcase
        end
    end
endmodule
```

2.1 What it does

Two requesters share one resource (in the final project, the OLED and UART). When both are waiting, the arbiter grants the one with the higher **effective priority**, which is its fixed base priority plus an aging bonus. The bonus grows by 1 for every AGE_THRESH cycles the request has been waiting, so a low-priority request eventually overtakes a high-priority one. A granted transfer is never interrupted. With aging_en set to 0 the bonus stays at 0 and the module becomes a plain fixed-priority arbiter, which gives the baseline for the before/after comparison without a second design.

2.2 Parameters and ports

Name	Type	Meaning
PRIO_W = 4	parameter	Base priority width, so priorities 0 to 15
BONUS_W = 8	parameter	Aging bonus width, so bonus 0 to 255
THR_W = 16	parameter	Threshold width, so up to 65,535 cycles per bonus step
EFF_W = 9	derived parameter	One bit wider than the larger of the two, so the sum cannot overflow. Do not override

Name	Type	Meaning
clk, rst	input	100 MHz clock, synchronous active-high reset
req[1:0]	input	Pending flags from the two requesters
done[1:0]	input	One-cycle completion pulses
grant[1:0]	output reg	One-hot grant, registered
aging_en	input	1 enables aging, 0 gives fixed priority
age_thresh	input	Cycles of waiting per +1 bonus (1 or more)
base_prio0, base_prio1	input	Base priorities
eff_prio0, eff_prio1	output	Current effective priorities, exposed for waveforms

2.3 Internal state

Register	Purpose
state	IDLE (deciding) or BUSY (a grant is active)
owner	Which channel currently holds the grant, so only its done releases it
last_served	Channel granted most recently, used by the tie-break. Resets to 1 so channel 0 wins the first tie
wait0, wait1	Cycles waited since the last bonus step
bonus0, bonus1	The aging bonus for each channel

2.4 Effective priority

The two assign lines zero-extend the base priority and the bonus to EFF_W bits and add them. The largest possible value is 15 + 255 = 270, which fits in 9 bits (maximum 511), so the sum can never wrap around.

2.5 Winner selection (combinational)

The always @* block decides who would win this cycle. Defaults are assigned first so no latch is inferred.

- If only one channel requests, it wins regardless of priority. The arbiter never idles while someone is waiting.
- If both request, the larger effective priority wins.
- On an exact tie, the winner is the channel that was **not** served last (~last_served). This means equal priorities alternate instead of one channel always winning, which scenario S4 verifies.

2.6 Aging counters

Each channel has the same logic. Its counters are cleared when any of three things is true: it is not requesting, it currently holds the grant, or aging is disabled. Otherwise the counter runs:

- Each cycle wait is incremented.
- When wait + 1 reaches age_thresh, wait restarts from 0 and the bonus rises by 1, stopping at BONUS_MAX (255).

The comparison is done in 17 bits so it cannot overflow, and a threshold of 0 behaves like 1. Aging keeps running while the **other** channel is being served, which is the whole point: waiting time during someone else's service counts toward promotion. With a threshold of 64 and a priority gap of 6, the low-priority channel's bonus reaches 6 after 384 waiting cycles.

2.7 Grant FSM

- **IDLE:** if any request is pending, register the one-hot grant for the winner, record it as owner and last_served, move to BUSY, and clear the winner's aging counters.
- **BUSY:** hold the grant until done[owner] pulses, then clear the grant and return to IDLE. A done pulse from the non-owner is ignored.

The aging block comes before the FSM in the same always block. With non-blocking assignments the last assignment to a register wins, so the FSM's clearing of the winner's counters overrides the aging update on the grant edge.

2.8 Cycle-level timing (from the simulation)

For a saturating requester with an 8-cycle service:

Cycle	Event
g	grant becomes visible
g + 9	the requester pulses done
g + 10	grant is low, arbiter is back in IDLE
g + 11	next grant becomes visible

So one service occupies the arbiter for SERVICE_CYCLES + 3 = 11 cycles. This is confirmed by the result: 60,000 / 11 is about 5,454, and S5 shows 5,455 grants. Request-to-grant latency has a floor of 1 cycle.

2.9 Design decisions worth defending

- **Non-preemptive:** SPI and I2C frames and an OLED or UART message cannot be aborted halfway, so a grant is never withdrawn early.
- **Registered grant:** gives clean timing for the downstream mux, at the cost of one cycle of latency.
- **Least-recently-served tie-break:** avoids a hidden fixed priority between equal-priority channels.
- **Owner-only release:** a stray done pulse cannot corrupt the state.
- **Cheap hardware:** per channel only one counter, one saturating counter and a comparator.

2.10 What is usually modified

Change	Where
Priorities	base_prio0, base_prio1 inputs (later from switches or registers)
Aging speed	age_thresh input, smaller means faster promotion
Aging off	aging_en = 0

Change	Where
Bonus cap	BONUS_W. The cap must stay larger than the biggest priority gap (15) or starvation protection is lost
More than two channels	Widen req, done, grant and replace the compare with a priority tree
Remove the idle cycle between services	Allow a re-grant in the same cycle as done
Non-linear aging	Change how the bonus grows, for example doubling steps

3. synth_requester.v (the synthetic interrupt generator)

```
`timescale 1ns/1ps
//
================================================================================
// synth_requester.v : SYNTHETIC INTERRUPT GENERATOR + fake sensor service
model
// (simulation only; not meant for synthesis)
//
// Behaves like a real sensor controller as seen by the arbiter:
//   raw irq event -> sticky pending (req) -> wait for grant -> "service" for
//   SERVICE_CYCLES -> done pulse -> req drops (or stays high if a new event
//   arrived during the service, so nothing is lost).
//
// MODE
//   0 periodic : one event every PERIOD cycles
//   1 random   : each cycle an event with probability ~PROB/65536 (16-bit
LFSR)
//   2 saturate : request is ALWAYS pending (re-requests the moment it
finishes)
//   3 one-shot : a single event at START_DELAY
//
// Event accounting (checked by the testbench every cycle):
//   irq_count == served_count + coalesced_count + req + repend
//   coalesced = event arrived while one was already pending (merged, like a
//   sticky interrupt flag); repend = event arrived during service (kept).
// Grant-to-done time = SERVICE_CYCLES + 1 cycles.
```

```
//
================================================================================
module synth_requester #(
    parameter integer MODE           = 0,
    parameter integer PERIOD         = 1000,
    parameter integer PROB           = 656,
    parameter integer SERVICE_CYCLES = 8,
    parameter integer START_DELAY    = 0,
    parameter [15:0]  SEED           = 16'hACE1
)(
    input  wire        clk,
    input  wire        rst,
    input  wire        grant,
    output reg         req,
    output reg         done,
    output reg         repend,
    output reg [31:0]  irq_count,
    output reg [31:0]  served_count,
    output reg [31:0]  coalesced_count
);
    reg [31:0] start_cnt, tmr, svc_cnt;
    reg [15:0] lfsr;
    reg        irq, serving, fired;

    wire active    = (start_cnt >= START_DELAY);
    wire finishing = serving && (svc_cnt >= SERVICE_CYCLES);

    always @(posedge clk) begin
        if (rst) begin
            start_cnt <= 0; tmr <= PERIOD-1; svc_cnt <= 0; lfsr <= SEED;
            irq <= 0; serving <= 0; fired <= 0;
            req <= 0; done <= 0; repend <= 0;
            irq_count <= 0; served_count <= 0; coalesced_count <= 0;
        end else begin
            done <= 1'b0;
            irq  <= 1'b0;
            lfsr <= (lfsr >> 1) ^ ({16{lfsr[0]}} & 16'hB400);
            if (start_cnt < START_DELAY) start_cnt <= start_cnt + 1;

            // ---------- raw interrupt source ----------
            case (MODE)
                0: if (active) begin
                       if (tmr >= PERIOD-1) begin tmr <= 0; irq <= 1'b1; end
```

```
                    else                        tmr <= tmr + 1;
                end
            1: if (active && (lfsr < PROB)) irq <= 1'b1;
            default: if (active && !fired) begin irq <= 1'b1; fired <=
1'b1; end
        endcase

        // ---------- service completes ----------
        if (finishing) begin
            done         <= 1'b1;
            serving      <= 1'b0;
            served_count <= served_count + 1;
            if (MODE == 2) begin                // saturating: instantly
pending again
                req       <= 1'b1;
                repend    <= 1'b0;
                irq_count <= irq_count + 1;
            end else begin
                req    <= repend | irq;
                repend <= 1'b0;
                if (irq) begin
                    irq_count <= irq_count + 1;
                    if (repend) coalesced_count <= coalesced_count + 1;
                end
            end
        end
        // ---------- ordinary interrupt arrival ----------
        else if (irq) begin
            irq_count <= irq_count + 1;
            if (serving) begin
                if (repend) coalesced_count <= coalesced_count + 1;
                else        repend <= 1'b1;
            end else if (req) coalesced_count <= coalesced_count + 1;
            else              req <= 1'b1;
        end

        // ---------- service start / progress ----------
        if (!serving && grant && req && !done) begin
            serving <= 1'b1;
            svc_cnt <= 1;
        end else if (serving && !finishing)
```

```
                svc_cnt <= svc_cnt + 1;
        end
    end
endmodule
```

3.1 What it does

It stands in for a real sensor controller. It produces raw interrupt events by one of four patterns, turns them into a sticky pending flag (req), waits for the arbiter's grant, pretends to read the sensor for a fixed number of cycles, then pulses done. It uses the same req, grant and done handshake the real ACL2 and HYGRO controllers will use, which is why the arbiter proven here can be reused unchanged.

3.2 Parameters

Parameter	Meaning
MODE	0 periodic, 1 random, 2 saturating, 3 one-shot
PERIOD	Cycles between events in periodic mode
PROB	In random mode, probability per cycle is about PROB / 65536. The tests use 30000 (about 46 percent) and 6000 (about 9 percent)
SERVICE_CYCLES	How long the fake sensor read takes. Grant-to-done is SERVICE_CYCLES + 1
START_DELAY	Cycles after reset before events start. Making two requesters share a delay makes their first requests simultaneous
SEED	Starting value of the 16-bit LFSR, so each requester gets a different but repeatable random stream

3.3 Internal state

Register	Purpose
start_cnt	Counts up to START_DELAY, then the requester becomes active

Register	Purpose
tmr	Period counter for periodic mode. Reset to PERIOD-1 so the first event fires immediately
svc_cnt	Counts service cycles while serving
lfsr	16-bit Galois shift register with taps 0xB400 (maximal length, 65,535 states). It never reaches zero if the seed is nonzero
irq	One-cycle registered raw interrupt pulse
serving	High from the cycle after grant is seen until the service finishes
fired	Makes saturating and one-shot modes fire their single initial event only once

3.4 Walkthrough of the always block

1. **Defaults.** done and irq return to 0 each cycle, which makes them one-cycle pulses. The LFSR advances every cycle.
2. **Event source.** Periodic mode counts tmr up and fires irq when it wraps. Random mode fires when the LFSR value is below PROB. Saturating and one-shot modes fire exactly one initial event.
3. **Service completion (finishing).** done is raised and the served counter increments. In saturating mode the request is immediately pending again, so req never drops. In the other modes req is set again only if a new event arrived during the service (repend) or at this exact cycle (irq).
4. **Ordinary interrupt arrival.** If an event comes while serving, it is remembered in repend, or merged if repend is already set. If it comes while a request is already pending, it merges (coalesces). Otherwise it sets req.
5. **Service start.** When grant is seen with a pending request, serving is set and svc_cnt starts counting. The !done term stops a restart in the cycle where done is high but grant has not yet dropped.

3.5 The event-accounting law

Every interrupt must be accounted for in exactly one place, so this holds at every clock edge:

```
irq_count = served_count + coalesced_count + req + repend
```

Term	Meaning
served	The event was read by the service
coalesced	It arrived while one was already pending and merged into it, like a sticky interrupt flag
req	It is the pending request right now
repend	It arrived during a service and is saved for the next one

The wrapper checks this every cycle. A violation would mean an interrupt vanished.

3.6 Why saturating mode keeps req high

If req dropped even for one cycle after done, the arbiter's single IDLE decision cycle could see only the low-priority request and grant it, so the low-priority channel would be served without aging. That would hide the starvation the experiment is meant to show. Keeping req continuously high makes the high-priority channel a true worst-case load.

3.7 What is usually modified

Change	Where
Faster or slower traffic	PERIOD or PROB
Different sensor speed	SERVICE_CYCLES. About 8 for a fast SPI read, 40 or more for a slow I2C read
Burst traffic	Add a mode that fires several events back to back, then pauses
Different random stream	SEED
Real hardware	Delete this module and connect adxl362_ctrl and hdc1080_ctrl to the same req, grant, done signals